Computers and Electrical Engineering 128 (2025) 110700

Contents lists available at ScienceDirect

Computers and Electrical Engineering

journal homepage: www.elsevier.com/locate/compeleceng

A shallow hybrid model with dynamic Bayesian optimisation for wind speed prediction on memory-constrained devices

Laeeq Aslam [a], Runmin Zou [a,*], Yaohui Huang [a], Fatima Yaqoob [b], Sharjeel Abid Butt [a], Qian Zhou [a]

[a] School of Automation, Central South University, Changsha, 410083, Hunan, China
[b] School of Geo-science and Info-physics, Central South University, Changsha, 410083, Hunan, China

ARTICLE INFO

Keywords:
Wind speed prediction
Hybrid model
Memory-constrained devices
Fast independent component analysis
Random vector functional link
Dynamic tree-structured Parzen estimator

ABSTRACT

Accurate wind speed prediction (WSP) supports energy management, but many deep learning models are too computationally intensive for embedded systems. This work presents a shallow hybrid model optimised with a dynamic tree-structured Parzen estimator. The model employs fast independent component analysis and an adaptive least absolute shrinkage and selection operator for feature selection, alongside a concurrent recurrent temporal module with a random vector functional link layer. The output is then refined through an attention layer to capture temporal dependencies. Across three sites, the model reduces mean squared error by up to 5.9% and mean absolute error by 5.03%, while increasing the coefficient of determination by 1.14 percentage points compared with the strongest baseline, alongside a 38.04% reduction in parameters. This design balances accuracy with resource efficiency and is well-suited to memory-constrained applications.

1. Introduction

Wind energy is a fundamental part of the renewable energy resources and plays a key role in global sustainability. It directly supports the United Nations Sustainable Development Goal 7 (SDG 7) by providing a clean and reliable power source. However, in energy management systems, a key challenge is the highly non-linear relationship between wind speed and available power, as shown in Eq. (1).

$$P = \frac{1}{2}\rho A y^3, \tag{1}$$

where P is the power output, ρ is air density, A is the rotor swept area and y is wind speed. For a given turbine with constant ρ and A, power output depends on the cubic function of y. This cubic relationship means that small changes in wind speed can lead to substantial variations in power output. Therefore, it is crucial to accurately predict wind speed for enhanced energy management and timely maintenance of wind turbines.

WSP methodologies fall into five approaches, each with limitations for short-term prediction. Physical models simulate wind using fluid dynamics but require substantial computational resources and capture non-linear dynamics poorly, making them unsuitable for real-time use [1]. Statistical models such as ARIMA, capture some non-linearities but struggle with the non-stationary nature of wind data [2]. Machine learning methods, including support vector regression and shallow neural networks, improve accuracy but are still constrained by simple architectures and too few parameters to model complex non-linear atmospheric interactions [3].

* Corresponding author.
E-mail address: rmzou@csu.edu.cn (R. Zou).

https://doi.org/10.1016/j.compeleceng.2025.110700
Received 15 November 2024; Received in revised form 2 September 2025; Accepted 2 September 2025
Available online 11 September 2025

These limits motivate architectures that learn non-linear wind-speed patterns efficiently and remain computationally efficient for short-term WSP.

Deep learning methods for time-series prediction, such as recurrent neural networks (RNN) and temporal convolutional networks (TCN), generally outperform statistical and physical approaches for WSP [4,5]. Long short-term memory networks (LSTM) and gated recurrent units (GRU), both RNN variants, address the vanishing-gradient problem and improve learning on sequences [6,7]. On the other hand, TCNs use causal convolutions to capture order without recurrence, which suits prediction tasks [8]. LSTM and GRU maintain an evolving hidden state that preserves long-range context, whereas TCNs capture hierarchical local patterns across a fixed dilated receptive field. In addition to recurrent and convolutional approaches, attention-based models such as Informer Flowformer and iTransformer show strong accuracy on meteorological time series [9–11]. They handle long contexts and often deliver stronger long-horizon accuracy, but their size and computational requirements hinder real-time WSP.

Building on these advances, hybrid models (HMs) are increasingly used to improve WSP accuracy. They apply feature extraction techniques such as empirical mode decomposition (EMD), variational mode decomposition (VMD), and wavelet transforms, then tune inputs and hyperparameters using genetic algorithms, particle swarm optimisation, Grey Wolf Optimisation, and the Tree-structured Parzen Estimator (TPE) [4,12]. The decomposition separates the wind series into high- and low-frequency bands, which supports cleaner temporal modelling [12]. Attention mechanisms further weight salient inputs [4]. Prior work used physics-informed vectors with LSTM and TCN to speed convergence and improve accuracy, but deployed LSTM and TCN separately without comparison to recent baselines [13]. Building upon this another study [14] combines the advection equation and a convolutional LSTM with an additive penalty loss to improve wind-speed prediction. Other work coupled LSTM with an improved Grey Wolf Optimisation to select correlated turbines and improved accuracy [15]. Random Vector Functional Link (RVFL) networks are widely used in hybrid models and deliver effective feature learning across prediction and classification tasks [16,17]. Their performance stems from hybrid representations and flexible parameterisation [18–20]. Ren et al. report improved short-term electricity load predicts with single-layer RVFLs that include direct input and output links and require minimal tuning [16]. Shiva et al. develop RVFL ensembles for online learning that update efficiently on sequential data and maintain real-time accuracy [19]. Gao et al. extend RVFLs to dynamic online ensembles that combine online decomposition with ensembling to handle distribution shift [20]. Despite these strengths, RVFLs offer limited adaptability and weak modelling of temporal dependencies, which restricts their use for time-series prediction.

Wind patterns vary by site owing to local climate, geography, terrain and built or natural obstructions. Models therefore require site-specific hyperparameter optimisation [7,21]. Comprehensive optimisation creates a large search space with discrete and continuous variables. This yields a mixed-integer non-linear programming problem where exact optimisation is intractable. Heuristic or Bayesian search can still deliver good suboptimal solutions. Popular methods such as Genetic Algorithm, Particle Swarm Optimisation, Grey Wolf Optimisation and the Tree-structured Parzen Estimator often slow down or converge prematurely in high dimensions. Recent variants report gains but typically tune only a few hyperparameters [15,22]. A complete treatment optimises feature selection, the input window length and core model settings, including layer widths, dropout and activation functions. Since regimes vary by site, choose a loss that fits local behaviour, for example Mean Squared Error(MSE) for abrupt shifts and Mean Absolute Error(MAE) for average error, and design the search to avoid premature convergence.

To address these challenges, we propose a dynamic tree-structured Parzen estimator-optimised shallow hybrid model (D-TPE-SHM) for wind speed prediction, designed for deployment on memory-constrained edge devices. The model first applies Fast Independent Component Analysis to extract informative features by isolating statistically independent components, which reduces redundancy in the inputs. These features then pass to a Concurrent Recurrent Temporal Module that learns diverse temporal patterns in parallel, overcoming the limits of cascaded hybrid designs. The CRTM output feeds a Random Vector Functional Link layer with an attention mechanism to capture additional latent temporal dependencies. To mitigate local minima and premature convergence, we introduce a Dynamic TPE scheme that searches an explicitly restricted hyperparameter space. We also define a balanced objective based on the harmonic mean of mean squared error and mean absolute error to stabilise accuracy across contrasting wind regimes such as shear and laminar flow. We test the model on a Jetson Nano and demonstrate high accuracy together with practical deployability. This paper makes three primary contributions

(1) This paper introduces a Dynamic Tree-Structured Parzen Estimator optimised shallow hybrid model (D-TPE-SHM) for wind speed prediction. The model couples Fast Independent Component Analysis with a Concurrent Recurrent Temporal Module to select relevant features and learn diverse temporal patterns, reducing redundancy and improving predictive accuracy.
(2) The Dynamic TPE module selects the model structure within a constrained hyperparameter space, reducing parameters and mitigating premature convergence. This is suited to memory-constrained edge devices while balancing accuracy and efficiency..
(3) We evaluate the model on real datasets and deploy it on a Jetson Nano, where it outperforms baseline models in efficiency, generalisability, and accuracy.

The rest of the paper is organised as follows: Section 2 covers the proposed methodology, Section 3 discusses the results and comparisons, and Section 4 concludes

2. Proposed methodology

This work presents a shallow hybrid model for wind speed prediction on memory-constrained edge devices. The architecture uses a single layer in each block to limit parameters and computation. Feature extraction applies Fast Independent Component

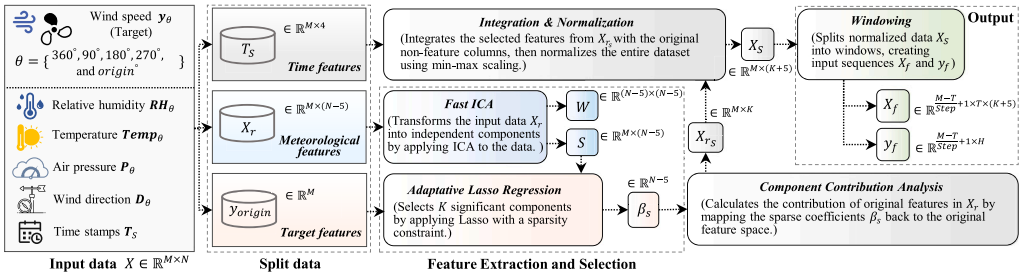


Fig. 1. Block diagram for feature selection mechanism using FastICA and adaptive Lasso regression.

Analysis followed by adaptive Least Absolute Shrinkage and Selection Operator (LASSO). Fast independent component analysis isolates independent components and reduces redundancy, while adaptive LASSO enforces sparsity and selects the most informative variables. The selected features are passed to a concurrent recurrent temporal module that runs a long short-term memory layer and a temporal convolutional network in parallel to learn long-range dependencies and local periodic structures. The module output is then fed to a random vector functional link layer with attention. The Random Vector Functional Link layer provides fast random projections and the attention focuses on salient temporal cues. To configure the model reliably under tight resource limits, this work tunes the hyperparameters with a dynamic tree-structured Parzen estimator. The optimiser balances exploration and exploitation and finds compact settings that preserve accuracy.

2.1. Feature selection using Fast Independent Component Analysis (FastICA) with adaptive Lasso regression

This study uses a spatio-temporal dataset $X \in \mathbb{R}^{M\times N}$ with M hourly records and N features. Each row $X_i \in \mathbb{R}^{1\times N}$ contains Year, Month, Day, Hour and five directional groups $F_{360^\circ}, F_{90^\circ}, F_{180^\circ}, F_{270^\circ}, F_{\text{origin}}$. For each direction F_{θ° the features are temperature (°C), relative humidity (%), wind speed ($\mathrm{m\,s^{-1}}$), wind direction (°) and air pressure (mbar). Hence each directional group lies in $\mathbb{R}^{1\times(N-4)/5}$ and the five groups together span $N-4$ features. The target variable $y_{\text{origin}} \in \mathbb{R}^{M}$ is the wind speed associated with F_{origin}. Fig. 1 summarises the feature selection pipeline with each block indicating a data transform and labelled outputs.

Feature selection begins by removing the Year, Month, Day, Hour and target y_{origin} columns from X, yielding $X_r \in \mathbb{R}^{M\times(N-5)}$. Fast independent component analysis (FastICA) is then applied to X_r to obtain the independent component matrix $S \in \mathbb{R}^{M\times(N-5)}$, computed as

$$S = W D^{-\frac{1}{2}} E' \left(X_r - \bar{X}_r\right), \tag{2}$$

where S denotes the independent components derived from X_r. W is the separation or mixing matrix. $D^{-\frac{1}{2}}$ is the inverse square root of the eigenvalues of the covariance of X_r. E' is the transpose of the corresponding eigenvectors. $\bar{X}_r$ is the mean of X_r used to centre the data. After FastICA the method applies adaptive Lasso to S to identify components that best predict y_{origin}. The aim is to minimise a Lasso objective with a sparsity constraint that keeps at least K non-zero entries in the coefficient vector $\beta_s \in \mathbb{R}^{(N-5)\times 1}$. The optimisation problem is

$$\min_{\beta_s} \frac{1}{2M} \|y_{\text{origin}} - S\beta_s\|_2^2 + \alpha \sum_{j=1}^{(N-5)} \frac{\beta_{s,j}}{\|\beta_{s,j}\|_1}, \tag{3}$$

where α is the sparsity parameter adjusted to ensure that exactly K coefficients remain non-zero. If this condition is not met, α is iteratively reduced until it is satisfied. Component Contribution Analysis (CCA) is then employed to compute the contribution of the original features in X_r:

$$\beta_{X_r} = A\beta_s, \tag{4}$$

where β_{X_r} denotes the coefficients for the original features in X_r and A is the inverse of W given by $A = W^{-1}$. This mapping identifies the features in X_r that contribute most to predicting y_{origin}. We select the top K features by the magnitude of β_{X_r} to form $X_{r_s} \in \mathbb{R}^{M\times K}$. We then build the final dataset $X_s = [\text{Year}, \text{Month}, \text{Day}, \text{Hour}, X_{r_s}, y_{\text{origin}}] \in \mathbb{R}^{M\times(K+5)}$. We normalise X_s with min–max scaling applied column wise. Next we apply windowing. T consecutive rows from all columns form the input $X_f \in \mathbb{R}^{((M-T)/\text{Step})+1\times T\times(K+5)}$ and the subsequent H targets form the output $Y_f \in \mathbb{R}^{((M-T)/\text{Step})+1\times H}$. The parameter Step controls the overlap between successive windows.

The final input X_f contains $((M-T)/\text{Step})+1$ samples, each with T time steps and $K+5$ features. The pipeline selects the most informative variables, including directional signals, and keeps only the top K for inference. For simplicity we use measurements from the four cardinal directions. Feature selection runs offline, so the extra cost is acceptable while deployment remains lightweight. We also assume neighbouring devices can share basic measurements, so predictions at the origin benefit from local context. The pseudo-code of the feature selection process is given in Algorithm 1.

Algorithm 1 Feature Selection Using FastICA and Adaptive Lasso

Require: Dataset $X \in \mathbb{R}^{M \times N}$, target feature count K, window size T
Ensure: Final processed dataset X_s, model inputs X_f, targets Y_f
$X_r \leftarrow X[:, \text{features}]$ {Remove non-feature columns (e.g., time, target)}
$S \leftarrow \text{FastICA}(X_r)$ {Apply FastICA (Eq. (2)) to get components}
$\alpha \leftarrow \alpha_0$ {initialise sparsity parameter}
while $\|\beta_s\|_0 < K$ **do**
  $\beta_s \leftarrow \arg\min_{\beta} \left\{ \frac{1}{2M} \|y_{\text{origin}} - S\beta\|_2^2 + \alpha\|\beta\|_1 \right\}$ {solve Lasso}
  if $\|\beta_s\|_0 < K$ **then**
    $\alpha \leftarrow \alpha - \delta\alpha$ {relax sparsity}
  end if
end while
$\beta_{X_r} \leftarrow A\beta_s$ {Map coefficients to original features (Eq. (4))}
$X_{r_s} \leftarrow X_r[:, \text{top-}K \text{ indices by } |\beta_{X_r}|]$ {Select top K features}
$X_s \leftarrow \text{min-max}(\text{concat}(\text{time cols}, X_{r_s}, y_{\text{origin}}))$ {Normalise final dataset}
$(X_f, Y_f) \leftarrow \text{window}(X_s, T)$ {Create sliding windows of size T}
return X_s, X_f, Y_f

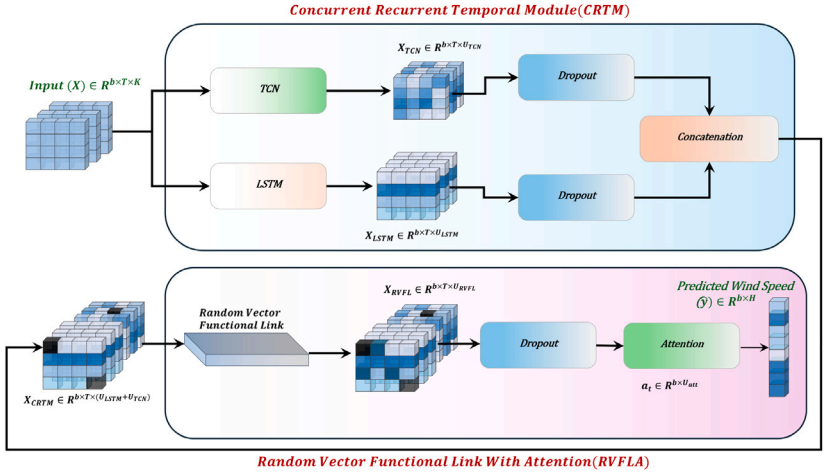


Fig. 2. Baseline architecture of the D-TPE-SHM.

2.2. Baseline architecture

The baseline comprises two modules. The first is a Concurrent Recurrent Temporal Module (CRTM). The second is a Random Vector Functional Link layer with attention (RVFLA). The CRTM runs an LSTM and a TCN in parallel to process spatio-temporal features. The RVFLA applies random projections to enrich the representation and uses attention to highlight the most informative components. A final dense layer maps the fused features to the prediction horizon. The structure and data flow are shown in Fig. 2.

2.3. Concurrent Recurrent Temporal Module (CRTM)

Once features are selected using FastICA and adaptive LASSO, the resulting windowed dataset forms the model input $\mathbf{X}_f \in \mathbb{R}^{\left(\frac{M-T}{\text{Step}}\right)+1 \times T \times K}$ and targets $\mathbf{Y}_f \in \mathbb{R}^{\left(\frac{M-T}{\text{Step}}\right)+1 \times H}$ for the baseline architecture. For notational simplicity, we subsequently write K in place of $K+5$. For training, we reserve 80% of the samples, yielding $\mathbf{X}_{ft}$ and $\mathbf{Y}_{ft}$ with $0.8\left(\frac{M-T}{\text{Step}}\right)+1$ examples. The CRTM comprises parallel LSTM and TCN branches, both operating on the same input $\mathbf{X}_{ft}$. With mini-batching, $\mathbf{X}_{ft}$ has shape $\mathbb{R}^{b \times T \times K}$, where b is the batch size. The input $\mathbf{X}_{ft}$ is processed by a single-layer LSTM with U_{LSTM} units to capture temporal dependencies. At each time step t, the hidden state $\mathbf{h}_t$ and cell state $\mathbf{c}_t$ are updated through the following equations:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_{ft} + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i), \tag{5}$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_{ft} + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f), \tag{6}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_c \mathbf{x}_{ft} + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c), \tag{7}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_{ft} + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o), \tag{8}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \tag{9}$$

Where, the input gate $\mathbf{i}_t$ in (5) controls how much of the current input $\mathbf{x}_{ft}$ at time t is written to the cell, while the forget gate $\mathbf{f}_t$ in (6) determines how much of the previous cell state $\mathbf{c}_{t-1}$ is retained. The cell state $\mathbf{c}_t$ in (7) therefore aggregates retained memory with new content. The output gate $\mathbf{o}_t$ in (8) then modulates the exposure of $\mathbf{c}_t$ to the hidden state, yielding $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$ in (9). This hidden state serves as the LSTM output at time t and encodes both short- and long-term dependencies. Under mini-batching, the sequence of hidden states has shape $\mathbb{R}^{b \times T \times U_{\text{LSTM}}}$. In the TCN block, the same $\mathbf{X}_{ft}$ is passed through a stack of causal, dilated one-dimensional convolutional layers with U_{TCN} filters, dilation rate Δ and kernel size K_s. The output at index t is given by (10):

$$\mathbf{y}_t = \sum_{k=0}^{K_s-1} \mathbf{W}_k * \mathbf{x}_{ft-k\Delta}, \tag{10}$$

where $*$ denotes discrete convolution and $\mathbf{W}_k$ are the kernel weights at offset k. By increasing the receptive field via K_s and Δ, the TCN captures long-range dependencies and hierarchical temporal structure, producing an output of shape $\mathbb{R}^{\text{b} \times T \times U_{TCN}}$. Dropout is applied to the LSTM and TCN outputs with rates D_{LSTM} and D_{TCN} to mitigate overfitting. The resulting features, $\mathbf{F}_{LSTM}$ and $\mathbf{F}_{TCN}$, are then concatenated along the last dimension to form the CRTM representation:

$$\mathbf{F}_{CRTM} = \text{Concat}(\mathbf{F}_{LSTM}, \mathbf{F}_{TCN}), \tag{11}$$

where $\mathbf{F}_{LSTM} \in \mathbb{R}^{\text{b} \times T \times U_{LSTM}}$ and $\mathbf{F}_{TCN} \in \mathbb{R}^{\text{b} \times T \times U_{TCN}}$. Consequently, $\mathbf{F}_{CRTM} \in \mathbb{R}^{\text{b} \times T \times (U_{LSTM}+U_{TCN})}$ jointly encodes sequential and hierarchical temporal patterns. The CRTM architecture accommodates both non-stationary and stationary behaviour. The LSTM branch adapts to non-stationary wind series and modulates its gates to track time-varying dependencies. The cell state offers persistent memory and supports recall of salient events across multiple time scales while handling irregular sequences. Several features follow seasonal or diurnal cycles, including temperature, humidity, wind direction and air pressure. The TCN branch captures these regularities with dilated causal convolutions that build hierarchical periodicity. Concurrent operation of the two branches outperforms a cascaded design with a TCN followed by an LSTM or GRU because early convolutional filtering can remove cues that matter for dynamic adaptation. Fusing the two outputs yields a complete and discriminative temporal representation and improves downstream prediction.

2.4. Random vector functional link layer with activation function

The Random Vector Functional Link (RVFL) layer receives the feature representation $\mathbf{X}_{CRTM} \in \mathbb{R}^{\text{b} \times T \times (U_{LSTM}+U_{TCN})}$ from the CRTM module after dropout. It applies a randomised weight matrix to introduce non-linear mappings to the input features without requiring gradient-based optimisation. A direct connection is then used to add the raw input features to the RVFL output, allowing both original and transformed features to contribute to the final output. Additionally, a bias term is included in both the random projections and the direct connection, enabling the model to learn shifts in the feature space. Let $\mathbf{W}_{RVFL} \in \mathbb{R}^{(U_{LSTM}+U_{TCN}) \times U_{RVFL}}$ denote the randomly initialised weight matrix in the RVFL layer, and $\mathbf{b}_{RVFL} \in \mathbb{R}^{U_{RVFL}}$ the bias vector for the random projection. First, the RVFL layer computes an intermediate output, denoted by $\mathbf{Y}$, using an activation function σ_{RVFL} that is applied element-wise to introduce non-linearity into the random projections:

$$\mathbf{Y} = \sigma_{RVFL}(\mathbf{X}_{CRTM} \mathbf{W}_{RVFL} + \mathbf{b}_{RVFL}), \tag{12}$$

where $\mathbf{Y} \in \mathbb{R}^{\text{b} \times T \times U_{RVFL}}$ represents the transformed output of the RVFL layer. The non-linear activation σ_{RVFL} allows the network to capture a broad range of patterns within the transformed feature space. The activation function σ_{RVFL} can be selected based on dataset characteristics, with common choices including ReLU, Sigmoid, Tanh, and Radbas. While the radial basis function (Radbas) has shown promising results in some cases, it may not fully leverage the concatenated features from LSTM and TCN in wind speed prediction, as these features already capture temporal and local patterns. ReLU is preferred here for handling non-linearities and avoiding vanishing gradients. Tanh and Sigmoid could also be beneficial, Tanh for its bounded output range, which aids stability, and Sigmoid for acting as a gating mechanism to focus on relevant features dynamically. The final output of the RVFL layer, $\mathbf{X}_{RVFL}$, is computed by concatenating the transformed output $\mathbf{Y}$ and the direct connection $\mathbf{X}_{CRTM}$. The RVFL layer applies a learnable transformation to $\mathbf{Y}$ through weights $\mathbf{W}_{out} \in \mathbb{R}^{U_{RVFL} \times U_{out}}$ and bias $\mathbf{b}_{out} \in \mathbb{R}^{U_{out}}$, producing $\mathbf{Z} = \mathbf{Y}\mathbf{W}_{out} + \mathbf{b}_{out}$. The final output is then:

$$\mathbf{X}_{RVFL} = \beta \times \left[\mathbf{Z}, \mathbf{X}_{CRTM}\right]. \tag{13}$$

Where, $\mathbf{Z} \in \mathbb{R}^{\text{b} \times T \times U_{out}}$ and $\mathbf{X}_{CRTM} \in \mathbb{R}^{\text{b} \times T \times (U_{LSTM}+U_{TCN})}$. The concatenated output $\mathbf{X}_{RVFL}$ has dimensions $\mathbb{R}^{\text{b} \times T \times U_{RVFL}}$, where $U_{RVFL} = U_{out} + U_{LSTM} + U_{TCN}$. In contrast to conventional ensemble deep Random Vector Functional Link (ed-RVFL), where β is computed using matrix inversion with regularisation, in this case, β are learnable weight. This structure allows the model to combine learned features from $\mathbf{Y}$ and raw input from $\mathbf{X}_{CRTM}$. The output $\mathbf{X}_{RVFL}$ is then passed through a dropout layer to mitigate overfitting. The output from this dropout layer maintains the same dimensions and is passed to the attention mechanism, which

uses Luong's multiplicative style to selectively emphasise temporal dependencies that are most relevant to the prediction task. In the attention mechanism, the last hidden state of the sequence $\mathbf{X}_{lt}$ is first extracted as given in Eq. (14)

$$\mathbf{X}_{lt} = \mathbf{X}_{RVFL}[:, -1, :], \tag{14}$$

where $\mathbf{X}_{lt} \in \mathbb{R}^{\mathrm{b}\times U_{RVFL}}$ represents the most recent temporal information captured by the sequence in each example. Each encoder hidden state $\mathbf{X}_{li} \in \mathbb{R}^{U_{RVFL}}$ within $\mathbf{X}_{RVFL}$ is then transformed by a learned weight matrix $\mathbf{W}_{\text{luong}} \in \mathbb{R}^{U_{RVFL}\times U_{RVFL}}$ to form $\mathbf{X}'_{li}$:

$$\mathbf{X}'_{li} = \mathbf{W}_{\text{luong}}\mathbf{X}_{li}, \tag{15}$$

where $\mathbf{X}'_{li} \in \mathbb{R}^{U_{RVFL}}$ represents each transformed hidden state in the sequence. The attention score for each time step i is calculated as the dot product between the last hidden state $\mathbf{X}_{lt}$ and each transformed hidden state $\mathbf{X}'_{li}$:

$$\text{score}_i = \mathbf{X}_{lt} \cdot \mathbf{X}'_{li}, \tag{16}$$

where $\text{score}_i \in \mathbb{R}^{\mathrm{b}\times T}$ represents the relevance of each timestamp i within the sequence. To obtain attention weights α_i, these scores are normalised using the softmax function:

$$\alpha_i = \frac{\exp(\text{score}_i)}{\sum_{k=1}^{T}\exp(\text{score}_k)}, \tag{17}$$

where $\alpha_i \in \mathbb{R}^{\mathrm{b}\times T}$ indicates the normalised importance of each hidden state in the sequence for generating the context vector. The context vector $\mathbf{c}$ is then computed as a weighted sum of the hidden states $\mathbf{X}_{li}$:

$$\mathbf{c} = \sum_{i=1}^{T}\alpha_i\mathbf{X}_{li}, \tag{18}$$

where $\mathbf{c} \in \mathbb{R}^{\mathrm{b}\times U_{RVFL}}$ serves as an aggregated representation of relevant temporal information across all time steps. The context vector $\mathbf{c}$ is concatenated with the last hidden state $\mathbf{X}_{lt}$ and passed through a dense layer with U_{att} units and tanh activation to produce the final attention output $\mathbf{a}_t$ as given in Eq. (19)

$$\mathbf{a}_t = \tanh(\mathbf{W}_c \cdot \text{concat}(\mathbf{c}, \mathbf{X}_{lt})), \tag{19}$$

where $\mathbf{a}_t \in \mathbb{R}^{\mathrm{b}\times U_{\text{att}}}$ represents the final output of the attention mechanism, encapsulating the essential temporal dependencies within the sequence for further processing. The tanh activation function bounds the output values, facilitating the learning of complex dependencies by focusing on both positive and negative contributions in the context vector. Finally, the output of this layer is projected onto a dense layer with units equal to the size of the horizon of prediction $\mathbf{H} \in \mathbb{R}^{\mathrm{b}\times U_{\mathrm{H}}}$.

2.5. Hyperparameter constraints and optimisation objective

The SHM defines a mixed hyperparameter space comprising continuous and discrete variables that govern feature selection, layer capacities, activation functions and dropout. We optimise these settings to strike a principled balance between large deviations and average error, acknowledging that short-term WSP often exhibits abrupt excursions driven by rapid pressure changes; a model tuned solely to penalise extremes will typically inflate mean error. Accordingly, the choice of inputs, loss function, and search bounds jointly target the simultaneous reduction of both error modes, while constraining model size to remain within the computational and memory budgets of edge hardware. Table 1 summarises the admissible ranges for each hyperparameter and Table 2 lists the loss functions considered. Input-related hyperparameters comprise the number of features K and the time window T, where K selects the most informative variables (Section 2.1) and T sets the span of historical context. Larger K and T capture broader spatial and temporal dependencies, albeit at increased computational cost in terms of floating point operations (FLOPs).

The baseline architecture exposes four hyperparameters, namely the units in the LSTM layer U_{LSTM}, the filters in the TCN layer U_{TCN}, the units in the RVFL layer U_{RVFL} and the units in the attention layer U_{att}. Large values inflate memory use and invite overfitting, whereas small values underfit and restrict representational power, so we optimise within bounded ranges to balance accuracy and efficiency. The TCN block adds two further controls that shape temporal coverage. The kernel size K_s sets the local receptive field for each convolution. Larger K_s can capture broader motifs but increases computation and the risk of overfitting, whereas smaller K_s economises compute but may miss longer dependencies. The dilation rate Δ spaces the kernel taps and expands the effective receptive field. Many studies keep Δ on a fixed exponential schedule such as $[1, 2, 4, 8, 16]$ to grow coverage in a regular way. We instead optimise Δ adaptively across TCN sublayers within problem-specific bounds so each sublayer selects its own rate. To preserve efficiency on constrained hardware, we cap both K_s and Δ as summarised in Table 1.

The remaining hyperparameters govern regularisation and non-linearity across the LSTM, TCN and RVFL blocks, namely the dropout rates $D_{\text{LSTM}}, D_{\text{TCN}}, D_{\text{RVFL}}$ and the activation functions $\sigma_{\text{LSTM}}, \sigma_{\text{TCN}}, \sigma_{\text{RVFL}}$. Dropout mediates the variance–bias trade-off: higher rates curb overfitting by stochastically discarding units, yet excessively high rates induce underfitting, while rates that are too low leave the model prone to memorisation. We therefore choose $D_{\cdot}$ within the bounds in Table 1 to balance generalisation and fidelity. The choice of $\sigma_{\cdot}$ shapes the learned representation. In the RVFL layer, a sigmoid acts as a soft gate that highlights salient projections, whereas tanh or ReLU introduces ungated non-linearity and expands the feature map. Within the LSTM cell, sigmoid activations regulate the input, forget, and output gates, while tanh defines the cell update, together controlling information flow over

Table 1
Constraints on hyperparameters for optimising memory usage and performance.

Architecture parameters			Regularisation & Functions		
Hyperparameter	Symbol	Range	Hyperparameter	Symbol	Range
Features	K	[1, 24]	LSTM units	D_{LSTM}	[0.1, 0.9]
Time window	T	[6, 48]	TCN dropout	D_{TCN}	[0.1, 0.9]
LSTM units	U_{LSTM}	[4, 256]	RVFL dropout	D_{RVFL}	[0.1, 0.9]
TCN units	U_{TCN}	[4, 256]	LSTM activation	σ_{LSTM}	{ReLU, Sig, Tanh}
Attention units	U_{att}	[4, 512]	TCN activation	σ_{TCN}	{ReLU, Sig, Tanh}
RVFL units	U_{RVFL}	[4, 512]	RVFL activation	σ_{RVFL}	{ReLU, Sig, Tanh, Radbas}
Kernel size	K_s	[1, 5]	Focal Loss α	α	[0.1, 5.0]
Dilation factor	Δ	[1, 128]	Focal Loss γ	γ	[0.1, 5.0]
			Loss function	L	{MSE, MAE, FL, QL, HL, LCL, SMAPE}

Table 2
Loss functions for regression head.

Loss function	Formula
Mean squared error	$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
Mean absolute error	$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$
Focal loss	$\text{FL} = \alpha(1-p_i)^{\gamma}\cdot\text{MSE} + \text{MAE},\ p_i = \exp(-\text{MSE})$
Quantile loss	$\text{QL} = \frac{1}{n}\sum_{i=1}^{n}\max(\tau\cdot(y_i-\hat{y}_i),(\tau-1)\cdot(y_i-\hat{y}_i))$
Huber loss	$\text{HL} = \begin{cases} 0.5(y_i-\hat{y}_i)^2, & \lvert y_i-\hat{y}_i\rvert \le \delta \\ \delta\cdot(\lvert y_i-\hat{y}_i\rvert - 0.5\cdot\delta), & \text{otherwise} \end{cases}$
Log-Cosh loss	$\text{LCL} = \sum\log(\cosh(\hat{y}_i - y_i))$
Symmetric mean absolute percentage error	$\text{SMAPE} = \frac{100}{n}\sum_{i=1}^{n}\frac{\lvert y_i-\hat{y}_i\rvert}{\lvert y_i\rvert+\lvert\hat{y}_i\rvert+1e-10}$

time. In the TCN, ReLU or tanh sustain gradient propagation and supports the capture of both short and long temporal dependencies. Calibrating these mechanisms per layer yields a compact model that learns rich structure without compromising robustness.

The final training hyperparameter is the loss function, which governs the trade-off between large errors and overall fit. Table 2 lists the available choices and their formulae and groups them into absolute-based, squared-based, and hybrid or specialised forms. Absolute-based losses such as MAE, Quantile Loss and SMAPE prioritise the average error and reduce sensitivity to outliers. Squared-based losses such as MSE, Huber and Log-Cosh place greater weight on large deviations and drive the model to suppress extremes. Hybrid forms such as Focal Loss blend these behaviours and allow targeted emphasis that reflects the data. Selection depends on local wind regimes and training goals. If winds remain relatively stable, an absolute-based loss like MAE often provides steady performance. If winds exhibit frequent sharp shifts, a squared-based loss can better control spikes. If regimes vary across months, a hybrid objective offers a balanced compromise. In practice we choose the loss alongside the inputs and other hyperparameters so that the model controls both average error and tail risk within device constraints. The objective function in Eq. (20), defined as the harmonic mean of MSE and MAE, aims to balance both large and average errors as follows:

$$\min_{\theta} OF(\theta) = \frac{2\cdot\text{MSE}(\theta)\cdot\text{MAE}(\theta)}{\text{MSE}(\theta)+\text{MAE}(\theta)}, \tag{20}$$

where the hyperparameter set θ is given as follows:

$$\theta = \{K, T, U_{\text{LSTM}}, U_{\text{TCN}}, U_{\text{att}}, U_{\text{RVFL}}, K_s, \Delta, D_{\text{LSTM}}, D_{\text{TCN}}, D_{\text{RVFL}}, \sigma_{\text{LSTM}}, \sigma_{\text{TCN}}, \sigma_{\text{RVFL}}, \alpha, \gamma, L\}. \tag{21}$$

Adjusting these hyperparameters within the constraints in Table 1 enables the model to balance large and absolute errors effectively, while remaining optimised for deployment on memory-limited devices.

2.6. Dynamic tree-structured Parzen estimator

The Tree-structured Parzen Estimator (TPE) is a Bayesian optimisation algorithm widely used for hyperparameter optimisation. TPE divides observed hyperparameter values into two groups, "good" and "bad", based on their objective function values and uses a threshold parameter, γ_{TPE}, to distinguish these groups. This approach enables TPE to balance exploration and exploitation during optimisation, achieving better performance over methods like grid search, Genetic Algorithms (GA) and Particle Swarm Optimisation (PSO) for hyperparameter optimisation tasks [23]. However, in its standard form, TPE has limitations, as it uses a fixed threshold of $\gamma_{\text{TPE}} = 0.2$. This fixed value may not be optimal for all objective functions, especially for those with complex, non-convex landscapes, where it may result in either excessive exploration in some areas of the hyperparameter space or premature convergence to suboptimal values. Thus, standard TPE may have slower convergence or achieve suboptimal points when applied to non-convex objective functions.

To address these issues, we propose a Dynamic TPE (D-TPE) approach, which dynamically adjusts γ_{TPE} based on the gradient of the objective function, OF(θ), where θ is a model hyperparameter. D-TPE starts with an initial threshold $\gamma_{\text{TPE}} = 0.2$ and updates it

within a range of $[0.1, 0.9]$ according to the gradient of $\mathrm{OF}(\theta)$. This adjustment allows γ_{TPE} to adapt to changes in the optimisation landscape, improving convergence speed and enabling D-TPE to achieve better suboptimal points compared to standard TPE. As shown in Eq. (22),

$$\gamma_{\mathrm{TPE}} = \gamma_{\mathrm{TPE}} - \eta \cdot \nabla \mathrm{OF}(\theta), \tag{22}$$

where γ_{TPE} is the threshold parameter, η is the learning rate and $\nabla \mathrm{OF}(\theta)$ represents the gradient of the objective function with respect to the hyperparameter θ. Here, η determines the adjustment magnitude for γ_{TPE}, allowing it to move in response to the gradient.

In this work, we implement two methods for adjusting the γ_{TPE} value in D-TPE: the Adam optimiser and the standard Gradient Descent approach. In the Adam-based adjustment, γ_{TPE} is updated using first and second-moment estimates of the gradient for a more controlled update. As shown in Eq. (23),

$$\gamma_{\mathrm{TPE}} = \gamma_{\mathrm{TPE}} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t, \tag{23}$$

where $\hat{m}_t$ and $\hat{v}_t$ are the first and second moment estimates, respectively, and ϵ is a small constant to ensure numerical stability. Adam calculates $\hat{m}_t$ and $\hat{v}_t$ as shown in Eqs. (24) and (25):

$$\hat{m}_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla \mathrm{OF}(\theta), \tag{24}$$

$$\hat{v}_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla \mathrm{OF}(\theta))^2, \tag{25}$$

where β_1 and β_2 are decay rates for the moment estimates and m_{t-1} and v_{t-1} are previous estimates of the first and second moments, respectively. The Adam optimiser uses these estimates to adjust γ_{TPE} by considering both the average gradient (momentum) and the variance, leading to stabilised updates. Alternatively, the Gradient Descent method in D-TPE adjusts γ_{TPE} using a direct gradient update rule. As shown in Eq. (26),

$$\gamma_{\mathrm{TPE}} = \gamma_{\mathrm{TPE}} - \eta \cdot \nabla \mathrm{OF}(\theta), \tag{26}$$

where η scales the gradient of $\mathrm{OF}(\theta)$, resulting in a straightforward adjustment of γ_{TPE} based on the current gradient without considering past gradients or variances. By dynamically adapting γ_{TPE} in response to the gradient, D-TPE can improve its convergence speed as well as achieve better suboptimal points for non-convex objective functions.

Algorithm 2 Dynamic TPE (D-TPE) Algorithm

Require: Initial $\gamma_{\mathrm{TPE}} = 0.2$, step size η, β_1, β_2, objective $\mathrm{OF}(\theta)$, ϵ
Ensure: Optimised γ_{TPE} and θ

Initialise $\hat{m}_0 \leftarrow 0$, $\hat{v}_0 \leftarrow 0$, $t \leftarrow 1$
while not converged **do**
  Compute $\nabla \mathrm{OF}(\theta)$
  if Adam update **then**
    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) \nabla \mathrm{OF}(\theta)$
    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) \big(\nabla \mathrm{OF}(\theta)\big)^2$
    $\hat{m}_t \leftarrow \dfrac{m_t}{1 - \beta_1^t} \quad \hat{v}_t \leftarrow \dfrac{v_t}{1 - \beta_2^t}$
    $\gamma_{\mathrm{TPE}} \leftarrow \gamma_{\mathrm{TPE}} - \dfrac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$
  else
    $\gamma_{\mathrm{TPE}} \leftarrow \gamma_{\mathrm{TPE}} - \eta \nabla \mathrm{OF}(\theta)$
  end if
  $\gamma_{\mathrm{TPE}} \leftarrow \min\big(\max(\gamma_{\mathrm{TPE}}, 0.1), 0.9\big)$
  Evaluate $\mathrm{OF}(\theta)$ with current γ_{TPE} and update θ if improved
  $t \leftarrow t + 1$
end while
return $\gamma_{\mathrm{TPE}}, \theta$

This dynamic adjustment allows D-TPE to balance exploration and exploitation more effectively across different regions of the hyperparameter space, improving optimisation performance compared to standard TPE. The D-TPE approach starts with an initial threshold parameter $\gamma_{\mathrm{TPE}} = 0.2$ and updates it based on the gradient of the objective function, $\mathrm{OF}(\theta)$. The pseudo-code in Algorithm 2 outlines the D-TPE process, including the initialisation and adjustment steps using Adam or Gradient Descent optimisers to dynamically adjust γ_{TPE}.

3. Results and comparisons

This section presents a comprehensive analysis of the D-TPE-SHM model's performance on three datasets, Chile, Kazakhstan and Mongolia. It details the experimental setup and dataset features, followed by comparisons across multiple performance metrics, such

Table 3
Dataset details and splits.

Location	Years	Total hours	Train (70%)	Validation (15%)	Test (15%)
Chile	2019 to 2022	35,064	24,545	5259	5260
Kazakhstan	2017 to 2019	26,280	18,396	3942	3942
Mongolia	2011 to 2015	43,824	30,677	6573	6574

as MSE, MAE, R^2 Score, Mean Absolute Percentage Error (MAPE), and Weighted Absolute Percentage Error(WAPE). Additionally, it provides a comparative assessment of different optimisers applied to the D-TPE-SHM model, including gradient-based D-TPE (G-D-TPE), Adam-based D-TPE (Adam-D-TPE), standard TPE, and a recently proposed hyperparameter optimisation method, improved grey wolf optimisation (SAGWO). This assessment focuses on the average rate of convergence (ARC), the optimal value of the objective function, and the area under the curve (AUC). A statistical analysis using the Wilcoxon Signed-Rank Test evaluates the effectiveness of D-TPE-SHM over TPE in terms of statistical significance. Finally, this section compares D-TPE-SHM's performance with state-of-the-art (SOTA), highlighting D-TPE-SHM's advantages in predictive accuracy and computational efficiency, which are critical for deployment on edge devices.

3.1. Experimental setup and dataset overview

This work uses hourly wind speed and meteorological data from the National Renewable Energy Laboratory. The sites are southern Chile (lat −53, lon −70.91), central Kazakhstan (lat 51.25, lon 73.42), and Ulaanbaatar, Mongolia (lat 47.93, lon 106.9). The selection targets regions with strong wind potential and practical need for small distributed wind systems that pair well with solar in local settings.

Each dataset provides 29 meteorological and temporal features, including temperature, humidity, wind speed, air pressure, and wind direction from the four cardinal points. Prior baselines report one-hour predicts with single-layer LSTM or TCN. This study predicts up to six hours ahead to support short-term operation. Table 3 summarises coverage and splits.

Hyperparameters are optimised offline with D-TPE. Each proposal trains for 20 epochs with patience 5, then evaluates the objective OF(θ) from MSE and MAE on held-out data. After the search budget, the top five settings by minimum OF(θ) train for 2000 epochs. Optimiser runtime is negligible relative to training time, so ARC and AUC serve to gauge consistency and convergence. For complete hyperparameter settings and sensitivity analyses, see the Supplementary Material.

3.2. Comparison metrics

This study evaluates model performance using a concise set of metrics. MSE and MAE appear in Table 2 and form part of the objective in Eq. (20). This section presents additional metrics for wind speed prediction. Their definitions follow.

$$\mathrm{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100\%, \tag{27}$$

$$\mathrm{WAPE} = \frac{\sum_{i=1}^{n}|y_i - \hat{y}_i|}{\sum_{i=1}^{n}|y_i|}, \tag{28}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \tag{29}$$

$$\mathrm{RMSPE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{y}_i}{y_i}\right)^2} \times 100\%. \tag{30}$$

The mean absolute percentage error (MAPE) reports the average percentage error. Additionally, the weighted absolute percentage error (WAPE) normalises the total absolute error by the sum of observed values and reduces bias when y approaches zero. Similarly, the coefficient of determination R^2 measures the proportion of variance that the model explains. In turn, the root mean square percentage error (RMSPE) squares percentage errors, so it weights larger relative errors more heavily and often increases at low wind speeds. Beyond accuracy, this study also assesses computational efficiency. Specifically, the floating point operations (FLOPs) count measures inference cost and a lower count supports edge deployment. Likewise, the total parameter count determines memory footprint and storage needs. In addition, the analysis compares optimisation behaviour using two metrics. First, the average rate of convergence captures how quickly the validation loss falls across epochs. Second, the area under the convergence curve (AUC) summarises cumulative progress during training.

$$\mathrm{ARC} = \frac{\mathrm{OF}(\theta)_i - \mathrm{OF}(\theta)_f}{\mathrm{NOI}}, \tag{31}$$

$$\mathrm{AUC} = \sum_{i=1}^{\mathrm{NOI}}\mathrm{OF}(\theta)_i. \tag{32}$$

Where, $\mathrm{OF}(\theta)_i$ and $\mathrm{OF}(\theta)_f$ denote the initial and final objective function values and NOI denotes the number of iterations. A higher ARC indicates faster convergence. A lower AUC indicates greater stability and efficiency during training.

Table 4
Performance evaluation of D-TPE-SHM model on Chile dataset across prediction horizons.

Horizon	MSE ↓	MAE ↓	R^2 ↑	MAPE ↓	WAPE ↓	RMSPE ↓
Horizon 1	0.0734	0.2039	0.9886	5.0243	3.4443	12.0279
Horizon 2	0.1962	0.3354	0.9695	7.8484	5.6713	15.9075
Horizon 3	0.4062	0.4835	0.9368	10.8415	8.1842	18.8409
Horizon 4	0.6976	0.6289	0.8913	13.9443	10.6279	25.4447
Horizon 5	1.0086	0.7569	0.8428	17.3359	12.7882	38.0623
Horizon 6	1.3363	0.8854	0.7922	20.2780	14.9717	41.9061
Average	0.6197	0.5490	0.9035	12.5454	9.2813	25.3649
Model GFLOPs ↓	0.0927			**Parameters (Million)** ↓		0.9088

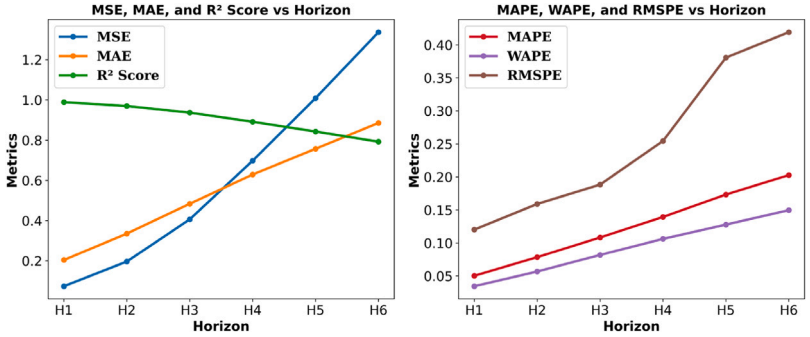


Fig. 3. Metric response to prediction horizon on Chile dataset.

3.3. Performance evaluation across datasets

The D-TPE-SHM model's performance is evaluated across six prediction horizons for each of the three datasets, Chile, Kazakhstan and Mongolia. Each dataset is assessed using MSE, MAE, R^2 Score, MAPE, WAPE and RMSPE, as summarised in Tables 4–6.

3.3.1. Chile dataset

The results for the Chile dataset are given in Table 4, demonstrating that the optimised hyperparameters yield overall low MSE and MAE across the six horizons, with average values of 0.6197 $(m/s)^2$ and 0.5490 (m/s), respectively. The model achieves a higher R^2 Score of 0.9035 on average, reflecting its ability to capture wind speed variance accurately. Moreover, the average relative error metrics, RMSPE, MAPE and WAPE, are 25.36%,12.5454% and 9.2813%, respectively. Furthermore, the optimisation of the model for the Chile dataset results in a model with 0.0927 giga FLOPS(GFLOPs) and 0.9088 million parameters. Since the model size was small enough, it was deployed on a Jetson Nano device after conversion to ONNX format.

Fig. 3 shows the behaviour of the metrics across different horizons. The MSE increases exponentially as the horizon extends, while the R^2 score decreases almost exponentially with the increase in the horizon. The other metrics, MAE, MAPE and WAPE, exhibit a nearly linear increase as the horizon progresses. This indicates that the model's ability to predict wind speed changes improves as we move closer to the event.

3.3.2. Kazakhstan dataset

Table 5 presents the D-TPE-SHM model's performance on the Kazakhstan dataset. The model demonstrates an average MSE of 0.446 $(m/s)^2$ and an MAE of 0.4758 (m/s), reflecting consistent accuracy across prediction horizons. The R^2 Score for Kazakhstan averages 0.893, indicating that the model successfully explains the majority of the wind speed variance. The relative error metrics, MAPE, WAPE and RMSPE average 19.22%, 13.86% and 30.02% respectively, which reflect the model's strong performance in percentage terms. The optimised model for Kazakhstan requires 0.094 GFLOPs and the number of parameters is 0.888 million. Despite its higher number of parameters compared to the optimised model for the Chile dataset, the model is still in feasible memory space as the selection was under constraints given in Table 1. Hence, the model still fits on the Jetson Nano when converted to ONNX format and the results presented here are computed on the Jetson device.

Fig. 4 shows a similar pattern. The MSE increases exponentially as the horizon extends, while the R^2 score decreases in the same manner. The other metrics, MAE, MAPE, RMSPE and WAPE, increase almost linearly as the horizon increases, indicating a consistent trend of higher error with increasing horizon.

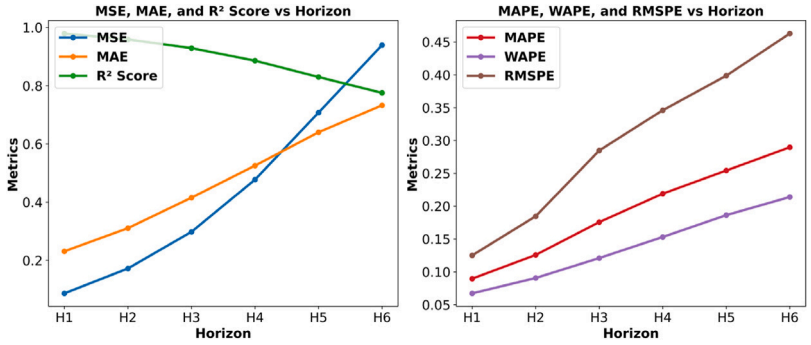


Fig. 4. Metric response to prediction horizon on Kazakhstan dataset.

Table 5
Performance evaluation of D-TPE-SHM model on Kazakhstan dataset across prediction horizons.

Horizon	MSE ↓	MAE ↓	R^2 ↑	MAPE ↓	WAPE ↓	RMSPE ↓
Horizon 1	0.0867	0.2310	0.9791	8.9562	6.7225	12.4956
Horizon 2	0.1722	0.3105	0.9588	12.5687	9.0622	18.4493
Horizon 3	0.2978	0.4152	0.9291	17.5590	12.0906	28.4651
Horizon 4	0.4771	0.5255	0.8860	21.8924	15.2954	34.6018
Horizon 5	0.7076	0.6399	0.8299	25.4225	18.6215	39.8675
Horizon 6	0.9395	0.7328	0.7755	28.9640	21.3830	46.2823
Average	0.4468	0.4758	0.8931	19.2271	13.8625	30.0269
Model GFLOPs ↓	0.094			**Parameters (Million)** ↓		0.888

Table 6
Performance evaluation of D-TPE-SHM model on Mongolia dataset across prediction horizons.

Horizon	MSE ↓	MAE ↓	R^2 ↑	MAPE ↓	WAPE ↓	RMSPE ↓
Horizon 1	0.0843	0.2011	0.9720	9.20	7.32	18.12
Horizon 2	0.1777	0.3055	0.9414	15.29	11.29	26.25
Horizon 3	0.3899	0.4452	0.8733	21.38	16.22	36.48
Horizon 4	0.5834	0.5541	0.8087	26.50	20.11	49.18
Horizon 5	0.7735	0.6310	0.7434	30.15	22.96	66.73
Horizon 6	0.9498	0.6936	0.6865	33.35	25.63	62.53
Average	0.4931	0.4717	0.8375	22.65	17.25	43.21
Model GFLOPs ↓	0.0465			**Parameters (Million)** ↓		0.8553

3.3.3. Mongolia dataset

As shown in Table 6, the Mongolia dataset results reveal an average MSE of 0.4931 $(m/s)^2$ and an MAE of 0.4717 (m/s), showing the model's lowest error margins among the three datasets. The average R^2 Score is 0.8375, indicating improved performance in explaining wind speed variance. The average MAPE and WAPE values are 22.65% and 17.25%, respectively, further confirming the model's accuracy in percentage terms. The Mongolia model operates efficiently with approximately 0.0465 GFLOPs and 0.8553 million parameters, which makes it possible to perform inference on the Jetson Nano device after converting the model to ONNX format.

Fig. 5 shows a similar pattern for the Mongolia dataset. The MSE increases exponentially with the horizon, while the R^2 score decreases almost exponentially. The other metrics, MAE, MAPE, RMSPE and WAPE, also increase in a nearly linear manner. This indicates that the first horizon has the least error, while the last horizon demonstrates the highest prediction error.

3.4. Comparative analysis of optimisers for hyperparameter tuning

This section evaluates four optimisation algorithms applied to the D-TPE-SHM model: the proposed gradient-based D-TPE (G-D-TPE), Adam-based D-TPE (ADAM-D-TPE), standard tree-structured Parzen estimator (TPE), and improved grey wolf optimisation (SAGWO) as proposed in [15].

The four optimisation algorithms (SAGWO, TPE, G-D-TPE and ADAM-D-TPE) are assessed on three primary metrics, ARC, Optimal value of OF(θ) and AUC. SAGWO demonstrates the lowest convergence rates and often converges prematurely, getting trapped in local minima. Although this algorithm has outperformed multiple other heuristic optimisers in [15], it fails to perform well in this case and is frequently stuck in local minima. This is because heuristic methods in high-dimensional problems require exponentially more computations for meaningful exploration. Whereas DTPE uses a probabilistic model to prioritise promising regions based on

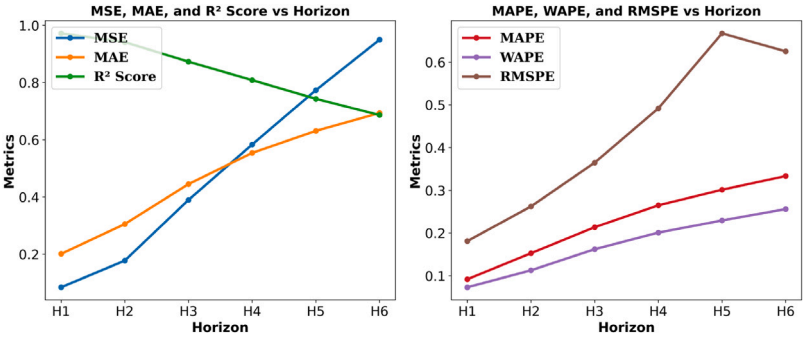
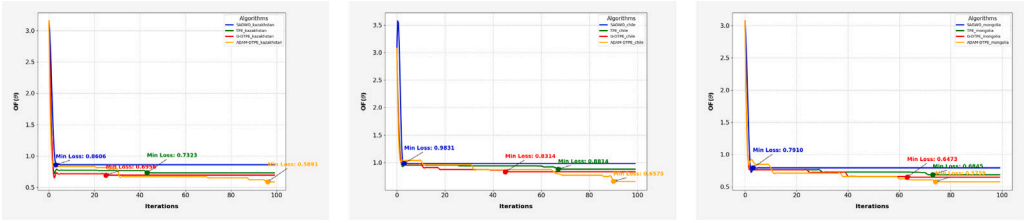


Fig. 5. Metric response to prediction horizon on Mongolia dataset.



(a) Loss vs. iteration for the Kazakhstan dataset across optimisation algorithms

(b) OF(θ) vs. iteration for the Chile dataset across optimisation algorithms

(c) Loss vs. iteration for the Mongolia dataset across optimisation algorithms

Fig. 6. Comparison across datasets and optimisation algorithms.

Table 7
Comparison of optimisation algorithms across datasets.

Algorithm	Chile			Kazakhstan			Mongolia		
	ARC ↑	Optimal OF(θ) ↓	AUC ↓	ARC ↑	Optimal OF(θ) ↓	AUC ↓	ARC ↑	Optimal OF(θ) ↓	AUC ↓
SAGWO	0.0217	0.9831	102.93	0.0229	0.8606	90.32	0.0236	0.7910	82.62
TPE	0.0227	0.8814	94.90	0.0242	0.7323	78.18	0.0247	0.6845	75.12
G-D-TPE	0.0232	0.8314	88.86	0.0246	0.6956	73.45	0.0251	0.6473	71.61
ADAM-D-TPE	**0.0242**	**0.6574**	**88.31**	**0.0257**	**0.5891**	**73.44**	**0.0255**	**0.5758**	**68.88**

a distribution, allowing TPE to perform better. In [15], only two hyperparameters (learning rate and number of hidden layers) were optimised, leading to strong performance. However, here we optimise 17 hyperparameters, making SAGWO less effective. In contrast, both G-D-TPE and ADAM-D-TPE show superior ARC with lower optimal OF(θ) values and lower AUC, showing their effectiveness at avoiding local minima, especially for complex hyperparameter tuning. Table 7 presents the ARC, Optimal OF(θ) and AUC for each algorithm across the Chile, Kazakhstan and Mongolia datasets, with the best values for each metric bolded.

For the Chile dataset, SAGWO achieves the lowest ARC of 0.0217 and reaches an optimal OF(θ) of 0.9831 with a high AUC of 102.93, indicating early stagnation in a local minimum. Compared to SAGWO, TPE shows improvement, with a higher convergence rate of 0.0227, lower optimal OF(θ) of 0.8814, and a better AUC of 94.90. Both G-D-TPE and ADAM-D-TPE outperform SAGWO and TPE, with G-D-TPE achieving the lowest optimal OF(θ) of 0.8314 at a convergence rate of 0.0232 and AUC of 88.86. However, ADAM-D-TPE provides the best results overall with the lowest optimal OF(θ) of 0.6574 and a convergence rate of 0.0242, showing it is the better optimiser in this case due to a more efficient convergence and better overall performance in minimising OF(θ). The performance of the optimisers for the Chile dataset is shown in Fig. 6(b), where G-D-TPE achieves its optimal solution before the 50th iteration, whereas ADAM-D-TPE, TPE, and SAGWO continue struggling until the last iteration. ADAM-D-TPE finds the best solution at the 90th iteration. Moreover, SAGWO gets stuck at local minima in the initial iterations.

In the Kazakhstan dataset, SAGWO reaches an optimal OF(θ) of 0.8606 with a convergence rate of 0.0229 and AUC of 90.32. TPE outperforms SAGWO, achieving an optimal OF(θ) of 0.7323 with a convergence rate of 0.0242 and AUC of 78.18. G-D-TPE slightly surpasses TPE with an optimal OF(θ) of 0.6956 and an almost same ARC of 0.0246, but its AUC is still lower than TPE at 73.45. Finally, ADAM-D-TPE delivers the best results, with the lowest optimal OF(θ) of 0.5891 at the highest convergence rate of 0.0257 and AUC of 73.44, showing its effectiveness in managing complex tuning with improved convergence, particularly in the high-dimensional context of this study. Iteration vs. OF(θ) values for the Kazakhstan dataset are shown in Fig. 6(a), where the red line for ADAM-D-TPE constantly reduces its value until the 90th iteration, whereas the rest of the optimisers get stuck around the 50th iteration in their local minima, resulting in suboptimal values.. The overall low optimal OF(θ) value for ADAM-D-TPE enables it to achieve a higher value of ARC.

Table 8
Descriptive statistics for the Chile dataset: D-TPE vs. TPE.

Metric	D-TPE					TPE				
	Mean/SD	Min/Max	25th	Median	75th	Mean/SD	Min/Max	25th	Median	75th
MSE ↓	0.62/0.15	0.35/0.97	0.52	0.57	0.66	0.84/0.23	0.50/1.53	0.67	0.78	0.93
MAE ↓	0.55/0.05	0.43/0.68	0.52	0.54	0.58	0.65/0.08	0.52/0.89	0.59	0.63	0.69
RMSE ↓	0.78/0.09	0.59/0.98	0.72	0.76	0.81	0.91/0.12	0.71/1.24	0.82	0.89	0.97
R^2 ↑	0.90/0.02	0.84/0.93	0.89	0.90	0.91	0.86/0.04	0.77/0.93	0.83	0.86	0.88

Table 9
Wilcoxon Signed-Rank Test results for Chile dataset (Corrected).

Metric	Wilcoxon Stat	P-value	Adjusted P-value	Significant
MSE↓	63.0	6.03×10^{-5}	3.01×10^{-4}	✓
MAE↓	34.0	1.73×10^{-6}	8.67×10^{-6}	✓
RMSE↓	60.0	4.40×10^{-5}	2.20×10^{-4}	✓
R^2 ↑	51.0	1.60×10^{-5}	8.02×10^{-5}	✓

Finally, for the Mongolia dataset, SAGWO, again with the lowest ARC of 0.0236, has an optimal OF(θ) of 0.7910 and a high AUC of 82.62, indicating limitations in escaping local minima. TPE achieves an ARC of 0.0247 with an optimal OF(θ) of 0.6845 and an AUC of 75.12, while G-D-TPE outperforms SAGWO and TPE with an ARC of 0.0251, a better optimal OF(θ) of 0.6473, and the lowest AUC of 71.61, showing its effectiveness for this dataset. However, ADAM-D-TPE, with an ARC of 0.0255 and optimal OF(θ) of 0.5758, performs even better. Moreover, its AUC is lower than the G-D-TPE, which shows its faster convergence towards the optimal result. Iteration vs. OF(θ) values for the Mongolia dataset are shown in Fig. 6(c). Although G-D-TPE achieves its sub-optimal value after 60 iterations, its optimised OF(θ) value is higher than ADAM-D-TPE's, which converges to the optimal value around the 75th iteration.

3.5. Comparative analysis of D-TPE and TPE performance using the Wilcoxon Signed-Rank Test

The Wilcoxon signed-rank test compares paired samples without assuming normality of the differences, so it suits small or non-Gaussian datasets and remains robust to outliers. This makes it well suited to hyperparameter optimisation, where score distributions often deviate from normality. Because D-TPE is a variant of TPE and long training can yield similar scores, the study tests whether any observed gain is reliable. The workflow holds out ten per cent of each dataset for testing, trains for up to 2000 epochs with a patience of 10 using the Adam optimiser with an adaptive learning rate, forms 32 independent partitions of the test data, and computes the mean, standard deviation, minimum, maximum, median, and the 25th and 75th percentiles for each metric. These summaries describe central tendency and variability. The Wilcoxon signed-rank test then compares the paired scores from D-TPE and TPE and returns p values that indicate whether the differences are statistically significant.

3.5.1. Chile dataset results

Table 8 shows that D-TPE gives lower means and standard deviation (SD) values than TPE. Mean MSE is 0.62 vs. 0.84 $(m/s)^2$, MAE is 0.55 vs. 0.65 (m/s) and RMSE is 0.78 vs. 0.91 (m/s), which indicates lower prediction error with D-TPE. The higher mean R^2 of 0.90 vs. 0.86 shows that the D-TPE model explains more variance. D-TPE adjusts the γ threshold in response to improvements in OF(θ) and this yields better hyperparameter choices than TPE.

Table 9 presents the Wilcoxon Signed-Rank Test results, which show statistically significant differences between D-TPE and TPE across all performance metrics. The p-values for each metric (MSE: 6.03×10^{-5}, MAE: 1.73×10^{-6}, RMSE: 4.40×10^{-5}, and R^2 Score: 1.60×10^{-5}) and the adjusted p-values confirm that the performance improvement observed with D-TPE over TPE is statistically significant, further supporting the effectiveness of the D-TPE method.

3.5.2. Kazakhstan dataset results

This section evaluates D-TPE's performance against TPE for the Kazakhstan dataset. Metrics such as MSE, MAE, RMSE and R^2 Score provide a detailed view of both algorithms' prediction accuracy and error rates. In Table 10, the descriptive statistics show that D-TPE provides lower MSE (0.4480 vs. 0.5948) $(m/s)^2$, MAE (0.4763 vs. 0.5561) (m/s) and RMSE (0.6626 vs. 0.7636) (m/s) values than TPE, indicating reduced prediction errors with D-TPE. The higher R^2 Score (0.8792 for D-TPE vs. 0.8439 for TPE) further indicates improved accuracy of D-TPE over TPE.

Furthermore, Table 11 shows the Wilcoxon Signed-Rank Test results, where significant p-values across all metrics indicate statistically reliable differences between D-TPE and TPE. The p-values (MSE: 1.4×10^{-5}, MAE: 3.5×10^{-6}, RMSE: 1.42×10^{-5} and R^2 Score: 1.95×10^{-4}) and their adjusted values provide strong evidence that D-TPE performs significantly better than TPE across both error and accuracy metrics.

Table 10
Descriptive statistics for the Kazakhstan dataset: D-TPE vs. TPE.

Metric	D-TPE					TPE				
	Mean/SD	Min/Max	25th	Median	75th	Mean/SD	Min/Max	25th	Median	75th
MSE ↓	0.4480/0.1274	0.1941/0.8374	0.3667	0.4299	0.5095	0.5948/0.1760	0.3560/1.2078	0.4441	0.5626	0.6632
MAE ↓	0.4763/0.0625	0.3321/0.6182	0.4272	0.4785	0.5107	0.5561/0.0698	0.4515/0.6994	0.5005	0.5381	0.5966
RMSE ↓	0.6626/0.0946	0.4406/0.9151	0.6056	0.6556	0.7137	0.7636/0.1079	0.5967/1.0990	0.6664	0.7501	0.8143
R^2 ↑	0.8792/0.0494	0.7444/0.9597	0.8586	0.8858	0.9154	0.8439/0.0523	0.6873/0.9175	0.8253	0.8530	0.8780

Table 11
Wilcoxon Signed-Rank Test results for Kazakhstan dataset.

Metric	Wilcoxon Stat	P-value	Adjusted P-value	Significant
MSE↓	50.0	1.424×10^{-5}	7.121×10^{-5}	✓
MAE ↓	39.0	3.512×10^{-6}	1.756×10^{-5}	✓
RMSE↓	50.0	1.424×10^{-5}	7.121×10^{-5}	✓
R^2 ↑	75.0	1.951×10^{-4}	9.754×10^{-4}	✓

Table 12
Descriptive statistics for the Mongolia dataset: D-TPE vs. TPE.

Metric	D-TPE					TPE				
	Mean/SD	Min/Max	25th	Median	75th	Mean/SD	Min/Max	25th	Median	75th
MSE ↓	0.49/0.16	0.28/1.24	0.41	0.48	0.52	0.64/0.25	0.35/1.71	0.48	0.60	0.68
MAE ↓	0.47/0.05	0.37/0.66	0.44	0.47	0.50	0.54/0.07	0.42/0.78	0.50	0.53	0.56
RMSE ↓	0.69/0.10	0.53/1.11	0.64	0.69	0.72	0.79/0.14	0.55/1.31	0.70	0.77	0.82
R^2 ↑	0.82/0.07	0.56/0.90	0.80	0.85	0.87	0.77/0.10	0.39/0.90	0.75	0.78	0.82

Table 13
Wilcoxon Signed-Rank Test results for Mongolia dataset.

Metric	Wilcoxon Stat	P-value	Adjusted P-value	Significant
MSE↓	17.0	9.64×10^{-8}	4.82×10^{-7}	✓
MAE↓	1.0	9.00×10^{-9}	4.70×10^{-8}	✓
RMSE↓	16.0	7.87×10^{-8}	3.94×10^{-7}	✓
R^2 ↑	20.0	1.73×10^{-7}	8.64×10^{-7}	✓

3.5.3. Mongolia dataset results

Finally, this section presents the results and comparisons for the Mongolia dataset for a detailed analysis of D-TPE's performance relative to TPE. In Table 12, the statistics indicate D-TPE's performance with lower MSE (0.49 vs. 0.64) $(m/s)^2$, MAE (0.47 vs. 0.54) (m/s), and RMSE (0.69 vs. 0.79) (m/s) values, suggesting improved prediction accuracy. Additionally, the higher R^2 Score for D-TPE (0.82) relative to TPE (0.77) further supports D-TPE's predictive advantage.

Table 13 presents the Wilcoxon Signed-Rank Test results, with p-values in scientific notation across all metrics, indicating statistically significant differences between D-TPE and TPE. The p-values (MSE: 9.64×10^{-8}, MAE: 9.00×10^{-9}, RMSE: 7.87×10^{-8}, R^2: 1.73×10^{-7}) confirm D-TPE's enhanced performance in terms of prediction accuracy.

3.6. Comparison of D-TPE-SHM with optimised state-of-the-art methods

Recently, several transformer-based models have been proposed for time series prediction. These include the Informer model, Flowformer, and iTransformer [9–11]. These models have demonstrated strong performance across diverse time series tasks, including weather-related datasets. However, despite their effectiveness, they tend to be computationally intensive and are generally designed for broader time series applications, which limits their adaptability when specifically fine-tuned for wind speed prediction. Additionally, the SAGWO-LSTM model introduced in [15] employs an adaptive grey wolf optimisation algorithm to enhance an LSTM network. Alongside SAGWO-LSTM, we compared the proposed model with the CoReSTL framework from [24], which leverages spatio-temporal data for wind speed prediction and aligns well with our dataset. Finally, we evaluated our model against ed-RVFL [17], an ensemble deep random vector functional link model designed for time series prediction. All results presented here are computed after optimising their hyperparameters using TPE, except for the SA-GWO model, which is optimised with the adaptive grey wolf algorithm proposed in the original manuscript [15].

Fig. 7 illustrates a visual performance comparison of D-TPE-SHM against state-of-the-art methods across the Chile, Kazakhstan, and Mongolia datasets. The spider chart presents normalised metric values, facilitating a clear and direct comparison among different models. Notably, D-TPE-SHM, depicted by the innermost circle (blue), demonstrates superior performance on the Chile dataset. For the Kazakhstan dataset, however, the proposed model shows inferior performance in terms of MAPE and marginally weaker performance in terms of FLOPs. Nevertheless, due to its significantly lower number of parameters compared to its closest competitor,

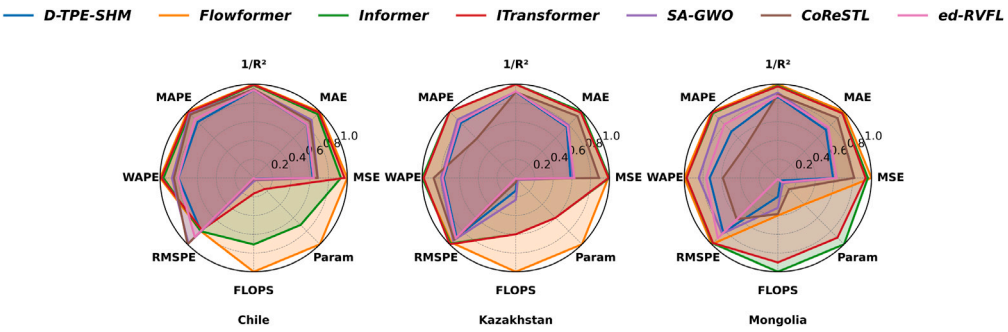


Fig. 7. Normalised performance comparison of D-TPE-SHM and state-of-the-art methods on Chile, Kazakhstan and Mongolia datasets.

Table 14
Performance comparison of D-TPE-SHM with SOTA models on the Chile dataset.

Model	MSE↓	MAE↓	$1/R^2$↓	MAPE↓	WAPE↓	RMSPE↓	FLOPS (G)↓	Params (M)↓
D-TPE-SHM	**0.620**	**0.549**	**1.107**	**12.55**	**9.28**	25.36	0.093	**0.909**
Flowformer [10]	0.984	0.682	1.181	14.83	11.55	25.47	5.140	75.602
Informer [9]	0.918	0.658	1.167	14.40	11.13	25.51	3.643	53.576
ITransformer [11]	0.956	0.673	1.175	14.61	11.39	**24.91**	0.858	12.625
SA-GWO [15]	0.674	0.597	1.117	14.36	10.09	31.38	0.192	2.711
CoReSTL [24]	0.671	0.582	1.117	14.20	9.84	31.80	0.050	2.001
ed-RVFL [17]	0.632	0.551	1.109	13.18	9.32	28.32	**0.002**	1.245

ed-RVFL, D-TPE-SHM maintains a lower overall size relative to all competing models. Lastly, for the Mongolia dataset, D-TPE-SHM underperforms compared to CoReSTL concerning the metrics MAPE, WAPE, and RMSPE but achieves better results in terms of MSE, MAE, and $1/R^2$ scores.

Despite exhibiting minor underperformance in certain metrics, D-TPE-SHM excels significantly in key metrics such as MSE, MAE, and the $1/R^2$ score. This demonstrates that the proposed optimised model offers a more balanced and robust approach overall. Although D-TPE-SHM exhibits higher FLOPs compared to ed-RVFL, this disadvantage is offset by having a considerably lower number of parameters, resulting in a smaller overall model size and making it a preferable option over ed-RVFL.

3.6.1. Performance on Chile dataset

Table 14 presents a detailed performance comparison between D-TPE-SHM and state-of-the-art (SOTA) models on the Chile dataset. D-TPE-SHM achieved an MSE of 0.62 $(m/s)^2$, the lowest among the models, indicating strong predictive accuracy on this dataset. In terms of MAE and $1/R^2$, D-TPE-SHM consistently outperformed other models, with values of 0.54 (m/s) and 1.10, respectively, demonstrating improved generalisation over more complex models such as Flowformer and Informer. Additionally, D-TPE-SHM's FLOPS and parameter counts were significantly lower than those of the transformer-based models, making it particularly suitable for deployment on memory-constrained devices like the Jetson Nano. However, ITransformer showed slightly superior performance in terms of RMSPE, and ed-RVFL had lower FLOPS, as it was not trained using backpropagation. Nevertheless, its number of parameters was significantly higher than that of the proposed model, resulting in a larger disk size compared to D-TPE-SHM.

3.6.2. Performance comparison on Kazakhstan dataset

In the Kazakhstan dataset results presented in Table 15, D-TPE-SHM achieves superior predictive accuracy, obtaining the lowest MSE of 0.44 $(m/s)^2$ and MAE of 0.47 (m/s), clearly outperforming the other evaluated models. Additionally, D-TPE-SHM attains lower values in key error metrics such as WAPE and RMSPE, as well as a competitive $1/R^2$ score, highlighting its capability to effectively minimise prediction errors. Although D-TPE-SHM exhibits higher computational complexity in terms of FLOPS compared to ed-RVFL, it utilises fewer parameters, demonstrating a more compact and optimised architecture. Thus, despite slightly increased computational costs, the significant improvements in critical performance indicators (MSE, MAE, $1/R^2$, WAPE, and RMSPE) justify the optimised model architecture and hyperparameter selections.

3.6.3. Performance on Mongolia dataset

For the Mongolia dataset, Table 16 shows that D-TPE-SHM achieves the lowest MSE and MAE, at 0.493 $(m/s)^2$ and 0.472 (m/s), respectively. In metrics like the $1/R^2$ score and number of parameters, this model also excels. However, it underperforms in terms of RMSPE, MAPE, and WAPE. Additionally, the low FLOPS and parameter count make D-TPE-SHM viable for deployment on devices like the Jetson Nano with memory limitations, as it allows for efficient processing in low-resource environments.

Hence, D-TPE-SHM consistently outperforms other SOTA models, achieving the lowest values in metrics such as MSE, MAE and $1/R^2$ Score across all three datasets. These results highlight the effectiveness of D-TPE-SHM in delivering high prediction accuracy

Table 15
Performance comparison of D-TPE-SHM with SOTA models on the Kazakhstan dataset.

Model	MSE↓	MAE↓	$1/R^2$↓	MAPE↓	WAPE↓	RMSPE↓	FLOPS (G)↓	Params (M)↓
D-TPE-SHM	**0.440**	**0.472**	**1.117**	20.95	**13.73**	**34.98**	0.094	0.888
Flowformer [10]	0.743	0.611	1.216	25.18	17.78	38.10	0.694	63.065
Informer [9]	0.752	0.613	1.219	25.23	17.83	38.62	0.417	37.875
ITransformer [11]	0.746	0.603	1.217	25.21	17.55	39.15	0.417	37.869
SA-GWO [15]	0.456	0.489	1.122	22.20	14.25	37.05	0.162	1.935
CoReSTL [24]	0.669	0.571	1.119	**14.93**	15.66	36.46	0.032	1.207
ed-RVFL [17]	0.471	0.479	1.127	21.44	13.95	35.53	**0.002**	**0.903**

Table 16
Performance comparison of D-TPE-SHM with SOTA models on the Mongolia dataset.

Model	MSE↓	MAE↓	$1/R^2$↓	MAPE↓	WAPE↓	RMSPE↓	FLOPS (G)↓	Params (M)↓
D-TPE-SHM	**0.493**	**0.472**	**1.195**	22.65	17.25	43.21	0.046	**0.855**
Flowformer [10]	0.837	0.650	1.381	32.25	23.66	52.41	0.093	8.458
Informer [9]	0.805	0.634	1.360	31.58	23.10	51.62	0.232	21.108
ITransformer [11]	0.784	0.633	1.348	31.91	23.06	51.36	0.209	19.010
SA-GWO [15]	0.525	0.492	1.252	28.89	19.99	44.07	0.074	1.620
CoReSTL [24]	0.683	0.586	1.229	**15.63**	**13.92**	**32.66**	0.089	3.560
ed-RVFL [17]	0.524	0.497	1.209	26.08	18.17	47.61	**0.002**	1.387

with minimal computational demands, particularly in environments with memory constraints, such as the Jetson Nano. Furthermore, D-TPE-SHM's efficiency in terms of parameter count makes it a practical choice for real-time applications on edge devices. The extensive hyperparameter optimisation, along with architectural refinements like FastICA for feature selection, concurrent TCN and LSTM processing and the RVFL integration, contributes to its superior performance. Overall, D-TPE-SHM provides an effective and resource-efficient solution for WSP, demonstrating a significant advancement over SOTA.

4. Conclusion

An optimised shallow hybrid model was developed to address the challenge of accurate short-term wind speed prediction on computationally constrained edge devices. The proposed methodology employed a two-stage process, integrating offline hyperparameter optimisation via a dynamic Tree-structured Parzen Estimator with the deployment of compact, site-specific models. The architecture incorporated Fast Independent Component Analysis and an adaptive LASSO for feature selection, a concurrent module blending long short-term memory and temporal convolutional networks to capture temporal dependencies, and a random vector functional link layer with an attention mechanism. Evaluation on operational data from sites in Chile, Kazakhstan, and Mongolia indicated consistent performance improvements against state-of-the-art benchmarks, with reductions in mean squared and absolute error observed alongside an increase in the coefficient of determination. These results were achieved within a computationally efficient framework of under 0.1 GFLOPs and one million parameters.

Future work will employ the proposed shallow hybrid model as a baseline to investigate its hyperparameter space across an extensive set of geographically distributed locations using the dynamic Tree-Structured Parzen Estimator. The mutual information between optimal hyperparameter configurations and the statistical distributions of site-specific input parameters will be systematically analysed. This investigation aims to establish correlations between data distribution characteristics and architectural hyperparameters. The resulting framework may substantially reduce the computational overhead associated with hyperparameter optimisation by enabling direct inference of suitable architectural parameters from input data statistics. This approach could facilitate rapid deployment to new sites without performing explicit hyperparameter optimisation, thereby reducing both setup time and computational requirements.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.compeleceng.2025.110700.

Data availability

Data will be made available on request.

References

[1] Liu Z, Ishihara T, He X, Niu H. LES study on the turbulent flow fields over complex terrain covered by vegetation canopy. J Wind Eng Ind Aerodyn 2016;155:60–73.
[2] Erdem E, Shi J. ARMA based approaches for forecasting the tuple of wind speed and direction. Appl Energy 2011;88(5):1405–14.
[3] Yang Q, Deng C, Chang X. Ultra-short-term/short-term wind speed prediction based on improved singular spectrum analysis. Renew Energy 2022;184:36–44.
[4] Wu J, Li N, Zhao Y, Wang J. Usage of correlation analysis and hypothesis test in optimizing the gated recurrent unit network for wind speed forecasting. Energy 2022;242:122960.
[5] Jiang W, Lin P, Liang Y, Gao H, Zhang D, Hu G. A novel hybrid deep learning model for multi-step wind speed forecasting considering pairwise dependencies among multiple atmospheric variables. Energy 2023;285:129408.
[6] Memarzadeh G, Keynia F. A new short-term wind speed forecasting method based on fine-tuned LSTM neural network and optimal input sets. Energy Convers Manage 2020;213:112824.
[7] Aslam L, Zou R, Awan ES, Hussain SS, Asim M, Chelloug SA, ELAffendi MA. Dynamic optimization of recurrent networks for wind speed prediction on edge devices. IEEE Access 2025;13:114520–41.
[8] Bai S, Kolter J, Koltun V. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. 2018, arXiv preprint arXiv:1803.01271.
[9] Zhou H, Zhang S, Peng J, Zhang S, Li J, Xiong H, Zhang W, Lin W. Informer: Beyond efficient transformer for long sequence time-series forecasting. In: Proceedings of the AAAI conference on artificial intelligence. vol. 35, 2021, p. 11106–15, 12.
[10] Wu H, Zhang H, Tian Q, Wang S, Hu T, Ma L, Long M. Flowformer: Linearizing transformers with conservation flows. 2022, arXiv preprint arXiv:2210.01726.
[11] Liu Y, Hu T, Zhang H, Wu H, Wang S, Ma L, Long M. iTransformer: Inverted transformers are effective for time series forecasting. 2023, arXiv preprint arXiv:2310.06625.
[12] Sun Z, Zhao M, Zhao G. Hybrid model based on VMD decomposition, clustering analysis, long short memory network, ensemble learning and error complementation for short-term wind speed forecasting assisted by Flink platform. Energy 2022;261:125248.
[13] Aslam L, Zou R, Awan E, Butt SA. Integrating physics-informed vectors for improved wind speed forecasting with neural networks. In: Proceedings of the 14th Asian Control Conference. 2024, p. 1902–7.
[14] Aslam L, Zou R, Huang Y, Awan ES, Butt SA, Zhou Q. Physics-informed spatio-temporal network with trainable adaptive feature selection for short-term wind speed prediction. Comput Electr Eng 2025;126:110517.
[15] Zhu A, Zhao Q, Yang T, Zhou L, Zeng B. Wind speed prediction and reconstruction based on improved grey wolf optimization algorithm and deep learning networks. Comput Electr Eng 2024;114:109074.
[16] Ren Y, Suganthan P, Srikanth N, Amaratunga G. Random vector functional link network for short-term electricity load demand forecasting. Inform Sci 2016;1078–93.
[17] Hu M, Chion JH, Suganthan PN, Katuwal RK. Ensemble deep random vector functional link neural network for regression. IEEE Trans Syst Man Cybern Syst 2022;53(5):2604–15.
[18] Zhang L, Suganthan P. A comprehensive evaluation of random vector functional link networks. Inform Sci 2016;1094–105.
[19] Shiva S, Hu M, Suganthan PN. Online learning using deep random vector functional link network. Eng Appl Artif Intell 2023;106676.
[20] Gao R, Li R, Hu M, Suganthan P, Yuen KF. Online dynamic ensemble deep random vector functional link neural network for forecasting. Neural Netw 2023;51–69.
[21] Aslam L, Zou R, Awan ES, Hussain SS, Shakil KA, Wani MA, Asim M. Hardware-centric exploration of the discrete design space in transformer–LSTM models for wind speed prediction on memory-constrained devices. Energies 2025;18(9):2153.
[22] He Y, Wang W, Li M, Wang Q. A short-term wind power prediction approach based on an improved dung beetle optimizer algorithm, variational modal decomposition, and deep learning. Comput Electr Eng 2024;116:109182.
[23] Abbas F, et al. Optimizing machine learning algorithms for landslide susceptibility mapping along the Karakoram highway, Gilgit Baltistan, Pakistan: A comparative study of Baseline, Bayesian, and metaheuristic hyperparameter optimization techniques. Sens 2023;23(15):6843.
[24] Yan B, Shen R, Li K, Wang Z, Yang Q, Zhou X, Zhang L. Spatio-temporal correlation for simultaneous ultra-short-term wind speed prediction at multiple locations. Energy 2023;284:128418.

Laeeq Aslam is a doctoral candidate at the School of Automation at Central South University. He works on machine learning for edge devices and sustainable energy systems. His interests include embedded deployment, renewable energy applications, time series prediction and data-driven modelling.

Runmin Zou is a Professor in the School of Automation at Central South University and leads work on smart energy and advanced control. His interests include control theory, artificial intelligence, power electronics, renewable energy and complex dynamical systems.

Yaohui Huang is a doctoral candidate at the School of Automation at Central South University. His research interests include time series forecasting, spatio-temporal data mining and load forecasting for energy systems.

Fatima Yaqoob is a doctoral candidate at the School of Geoscience and Information Physics at Central South University. Her interests include spatio-temporal data analysis, geostatistics and predictive modelling for geological and geophysical datasets.

Sharjeel Abid Butt is a doctoral candidate at the School of Automation at Central South University. His interests include machine learning, data science, automation and embedded systems with an emphasis on practical deployment.

Qian Zhou is a master 's student at the School of Automation at Central South University. Her interests include convolutional and recurrent neural networks, model evaluation for regression, electric vehicle health assessment and energy-related applications of machine learning.